

Stellar Trace: Host Profiling & Machine Learning

Session 2: Class Imbalance, Data Leakage, & Neural Network Classifiers
(Tasks 10–12)

Astronomy Club

July 17, 2026

Phase 2

Stellar Trace Phase 2 transitions from the volume-complete universe model of Milestone 1 to real data ingestion, classifier training, and physical model fitting:

Session 1: Telescopes & Statistics (Tasks 7–9)

- Selection effects, coordinate cross-matching, database querying, and 1D/2D K-S tests.

Session 2 (Today): Machine Learning & Data Leakage (Tasks 10–12)

- Task 10: Addressing Class Imbalance via Dataset Oversampling.
- Task 11: Multiclass Neural Network & Hyperparameter Optimization.
- Task 12: Model Interpretability via Permutation Importance & OOP Assembly.

Session 3: Rate Fitting & Physics-Informed nets (Tasks 13–15)

- Bayesian MCMC (Metropolis-Hastings), PINNs, and DTD Convolution.

Session 2 Objectives

By the end of this session, you will be able to:

1. Explain the class imbalance problem in transient astronomy and why accuracy is a misleading metric.
2. Identify the mechanisms of data leakage and implement a leak-free oversampling pipeline.
3. Explain the mathematics of Multi-Layer Perceptrons (MLPs), hidden layers, non-linear activations, and Softmax output.
4. Derive the gradient of the cross-entropy loss function with respect to neural network logits.
5. Optimize neural networks using 5-fold cross-validation grid searches over hyperparameter grids.
6. Evaluate multiclass models using Macro F1-scores, Confusion Matrices, and One-vs-Rest ROC/AUC curves.
7. Calculate Permutation Feature Importance to extract physical rules learned by neural networks.
8. Refactor modular science scripts into a production-grade Object-Oriented pipeline class.

Section 1: Class Imbalance & Data Leakage (Task 10)

The Class Imbalance Problem in Astronomy

Cosmic explosions (supernovae, gamma-ray bursts, fast radio bursts) are rare physical anomalies. Consequently, host galaxy catalogs are orders of magnitude smaller than catalogs of the general field galaxy population.

Our Compiled Machine Learning Dataset

- **Class 0 (Background field galaxies):** $N_0 = 150$ (Majority class)
- **Class 1 (Core-Collapse SN hosts):** $N_1 = 30$ (Minority class)
- **Class 2 (Type Ia SN hosts):** $N_2 \approx 17 - 20$ (Minority class, from VizieR matching)

Why Global Accuracy Fails

If a model is trained directly on this imbalanced dataset, the loss function will optimize performance on the majority class. A naive classifier predicting Class 0 for every galaxy yields an accuracy of:

$$\text{Accuracy} = \frac{150}{150 + 30 + 20} = 75\%$$

despite failing to identify a single supernova host environment. We must rebalance the data.

Class Imbalance Math: Precision, Recall, and F1

To evaluate imbalanced classifiers, we look at performance on each class c individually:

- **Precision (P_c):** Out of all predicted instances of class c , how many actually belong to c ?

$$P_c = \frac{TP_c}{TP_c + FP_c}$$

- **Recall (R_c):** Out of all actual instances of class c , how many did we successfully find?

$$R_c = \frac{TP_c}{TP_c + FN_c}$$

- **F1-score ($F1_c$):** The harmonic mean of Precision and Recall. It balances both metrics:

$$F1_c = 2 \cdot \frac{P_c \cdot R_c}{P_c + R_c} = \frac{2TP_c}{TP_c + \frac{1}{2}(FP_c + FN_c)}$$

- **Macro F1-score ($F1_{\text{macro}}$):** The unweighted arithmetic average of F1-scores across all classes:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c$$

Treats rare classes (SNe hosts) with the same importance as background

Oversampling and Rebalancing Strategies

How do we mathematically balance our training data?

Undersampling: Discarding majority class instances (Class 0) until class sizes are equal.

- **Drawback:** Discards valuable physical information, reducing the dataset size.

Random Oversampling (ROS): Sampling minority class instances (Class 1 and 2) with replacement until they match the majority class.

- Let S_c be the training samples of class c , and $N_{\text{target}} = \max_c |S_c|$.
- If $|S_c| < N_{\text{target}}$, draw new samples uniformly with replacement:

$$x^* \sim \mathcal{U}(S_c) \quad \text{until} \quad |S_c| = N_{\text{target}}$$

- **Drawback:** Can lead to overfitting because the model memorizes exact duplicates.

Synthetic Minority Over-sampling (SMOTE): Generates synthetic instances along the line segments connecting minority class data points and their nearest neighbors:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim U(0, 1)$$

The Fatal Trap: Data Leakage

Applying oversampling to the entire dataset **prior** to performing the train-test split is a critical methodological error known as **data leakage** (or validation contamination).

How Data Leakage Occurs

The minority classes are oversampled (via duplication or SMOTE), generating identical or near-identical instances.

The dataset is split into training and test sets.

Identical clones of the same galaxies are distributed into both the training and test sets.

During training, the classifier simply memorizes the clones. It evaluates them on the test set, achieving an artificially perfect classification score:

$$\text{F1-score} \approx 1.00$$

Physical Reality Check: When deployed on new telescope observations, this model fails catastrophically because it memorized duplicates rather than learning physical features.

The Leakage-Free Pipeline

To prevent data leakage, we must strictly isolate the test set from any oversampling procedures:

Correct Order of Operations

1. **Stratified Split:** Split the raw dataset into training (70%) and test (30%) sets. We use a stratified split to ensure the proportions of Class 0, 1, and 2 are identical in both sets.
2. **Isolate Test Set:** Place the test set aside. It remains untouched, imbalanced, and scale-free, representing the true physical distribution of the universe.
3. **Oversample Training Set:** Apply oversampling **only** to the training data.
4. **Train & Evaluate:** Train the model on the rebalanced training set. Evaluate on the untouched, imbalanced test set.

Validation Principle

Always validate on the true physical distribution. The test set must remain as messy as the real universe.

Task 10: Conceptual Recap

In `stellar_trace_assignment2.ipynb`, you implement the stratified split and training-only manual oversampling function:

- **Stratified Splitting:** Utilizes `sklearn.model_selection.train_test_split` with the parameter `stratify=df_ml['class']` to enforce identical class distributions in the training (70%) and testing (30%) subsets.
- **Manual Oversampling:** Rather than importing external libraries (like `imblearn`), you write a custom function `oversample_dataset`:
 1. Identifies the maximum class count in the training partition ($N_{\text{target}} = 105$).
 2. Iterates over each class subset.
 3. For minority classes (1 and 2), samples with replacement (`replace=True`) until the size equals N_{target} .
 4. Concatenates the balanced partitions back into a unified training set.
- **Verification:** Confirms that the training set becomes perfectly balanced (105 instances per class), while the test set remains imbalanced and untouched (45 field, 9 CC-SNe, 6 Ia-SNe hosts).

Section 2: Multi-Layer Perceptrons & Neural Net Math (Task 11)

What is a Multi-Layer Perceptron (MLP)?

An MLP is a feed-forward artificial neural network that maps an input feature vector $\mathbf{x} \in \mathbb{R}^d$ to a target probability distribution over C classes.

- **Input Features ($d = 4$):**

$$\mathbf{x} = [\text{redshift}, \log_{10} M_{\star}, \log_{10} \text{SFR}, \text{ssfr}]^T$$

- **Network Architecture:**

- **Input Layer:** Receives the normalized physical features.
 - **Hidden Layers:** Performs non-linear transformations to extract complex parameter boundary shapes (corridors).
 - **Output Layer:** Outputs class probabilities (Class 0, 1, or 2).
- **Normalizing Inputs (StandardScaler):** Because features have wildly different physical units (redshift ~ 0.1 , Mass $\sim 10^{10} M_{\odot}$), we must normalize features using a StandardScaler to ensure stable gradient descent:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

Hidden Layer Mathematics

Each neuron in a hidden layer performs a linear combination of its inputs, adds a bias, and applies a non-linear activation function:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

Where:

- $\mathbf{a}^{(l-1)}$ is the activation vector from the previous layer ($\mathbf{a}^{(0)} = \mathbf{x}_{\text{scaled}}$).
- $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weight matrix and bias vector of layer l .
- $f(z)$ is the **activation function**.

Layer Dimensions for `hidden_layer_sizes = (32, 16)`

- **Input:** $\mathbf{a}^{(0)} \in \mathbb{R}^4$
- **Hidden Layer 1:** $\mathbf{W}^{(1)} \in \mathbb{R}^{32 \times 4}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{32}$, output $\mathbf{a}^{(1)} \in \mathbb{R}^{32}$
- **Hidden Layer 2:** $\mathbf{W}^{(2)} \in \mathbb{R}^{16 \times 32}$, $\mathbf{b}^{(2)} \in \mathbb{R}^{16}$, output $\mathbf{a}^{(2)} \in \mathbb{R}^{16}$
- **Output Layer:** $\mathbf{W}^{(3)} \in \mathbb{R}^{3 \times 16}$, $\mathbf{b}^{(3)} \in \mathbb{R}^3$, output logits $\mathbf{z}^{(3)} \in \mathbb{R}^3$

Hidden Layer Activation Functions

Why Non-Linear Activations?

Without a non-linear function $f(z)$, nested linear transformations collapse into a single linear regression ($\mathbf{W}_3\mathbf{W}_2\mathbf{W}_1\mathbf{x} + \mathbf{b}'$), preventing the network from learning complex shapes (like the bimodal main sequence).

ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

- **Range:** $[0, \infty)$
- **Derivative:** $f'(x) = 1$ if $x > 0$, else 0.
- **Pros:** Fast computation, mitigates vanishing gradients.
- **Cons:** “Dying ReLU” problem (neurons stuck in the zero-gradient region).

tanh (Hyperbolic Tangent):

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Range:** $(-1, 1)$
- **Derivative:** $f'(x) = 1 - \tanh^2(x)$
- **Pros:** Zero-centered, which stabilizes weight updates.
- **Cons:** Suffers from vanishing gradients for large $|x|$.

Output Layer & Softmax Activation

The final layer of the network contains $C = 3$ output nodes, corresponding to Class 0, 1, and 2. It produces raw real-valued numbers called **logits** $\mathbf{z} \in \mathbb{R}^3$.

The Softmax function

To turn logits into probabilities, we use the Softmax function:

$$P_c = P(y = c \mid \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=0}^{C-1} e^{z_j}}$$

- Softmax exponentiates each logit, ensuring they are positive.
- It divides by the sum of all exponentiated logits, normalizing the output such that they sum to exactly 1.0:

$$\sum_{c=0}^2 P(y = c \mid \mathbf{x}) = 1.0$$

- The model's final prediction is the class with the highest probability:

$$\hat{y} = \operatorname{argmax}_c P_c$$

Loss Function: Categorical Cross-Entropy

To train the network, we measure the error between predicted probabilities and the true classes using the **Categorical Cross-Entropy Loss** ($\mathcal{L}$).

Loss for a single galaxy i

$$\mathcal{L}_i = - \sum_{c=0}^{C-1} y_{ic} \log P_{ic}$$

where y_{ic} is a one-hot encoded indicator (1 if class c is the true label, 0 otherwise).

Average Loss with L2 Regularization (Weight Decay)

To prevent overfitting by keeping weights small, we add an L2 penalty on the weights:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=0}^{C-1} y_{ic} \log P_{ic} + \frac{\alpha}{2} \sum_{l=1}^L \sum_p \sum_q (W_{pq}^{(l)})^2$$

where α is the weight decay hyperparameter.

Derivation: Logit Gradients (Part 1 - Setup)

The error gradient with respect to the output logits $\mathbf{z}$ is critical for backpropagation. Let's derive $\frac{\partial \mathcal{L}_i}{\partial z_c}$ for a single galaxy. By the Chain Rule, we expand the derivative of the loss function:

$$\frac{\partial \mathcal{L}_i}{\partial z_c} = - \sum_{k=0}^{C-1} y_k \frac{\partial \log P_k}{\partial z_c} = - \sum_{k=0}^{C-1} y_k \frac{1}{P_k} \frac{\partial P_k}{\partial z_c}$$

Recall that the Softmax output for class k is $P_k = e^{z_k} / \sum_j e^{z_j}$. Its derivative with respect to logit z_c is:

$$\frac{\partial P_k}{\partial z_c} = P_k (\delta_{kc} - P_c)$$

where Kronecker's delta $\delta_{kc} = 1$ if $k = c$, and 0 if $k \neq c$.

Derivation: Logit Gradients (Part 2 - Simplification)

Substituting the Softmax derivative back into the Chain Rule expansion:

$$\begin{aligned}\frac{\partial \mathcal{L}_i}{\partial z_c} &= - \sum_{k=0}^{C-1} y_k \frac{1}{P_k} [P_k(\delta_{kc} - P_c)] \\ &= - \sum_{k=0}^{C-1} y_k (\delta_{kc} - P_c) \\ &= - \sum_{k=0}^{C-1} y_k \delta_{kc} + P_c \sum_{k=0}^{C-1} y_k\end{aligned}$$

Since $\mathbf{y}$ is one-hot encoded, $\sum_{k=0}^{C-1} y_k = 1.0$. And $\sum_k y_k \delta_{kc} = y_c$.

$$\frac{\partial \mathcal{L}_i}{\partial z_c} = P_c - y_c$$

The Gradient Rule

The logit gradient is simply the linear prediction error: $P_c - y_c$.

L2 Weight Decay Gradient Update

How does the L2 penalty affect our gradient descent updates?

- Let $\mathcal{L}_0$ be the unregularized loss, and $\mathcal{L} = \mathcal{L}_0 + \frac{\alpha}{2} W^2$ (for a single weight W).
- Taking the partial derivative with respect to W :

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}_0}{\partial W} + \alpha W$$

- The gradient descent update rule with learning rate η is:

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}$$

- Substituting the derivative:

$$W \leftarrow W - \eta \left(\frac{\partial \mathcal{L}_0}{\partial W} + \alpha W \right) = (1 - \eta\alpha)W - \eta \frac{\partial \mathcal{L}_0}{\partial W}$$

Weight Decay Interpretation

At each step, we multiply the weight by a factor $(1 - \eta\alpha) < 1.0$ before applying the standard gradient step. This shrinks weights toward zero, reducing model complexity!

Section 3: Model Evaluation & Hyperparameter Optimization (Task 11)

Hyperparameter Tuning via K-Fold Cross-Validation

We search for the best network settings using **5-Fold Cross-Validation** over a grid of hyperparameters:

- **Why Cross-Validation?** Evaluating only on a single split can lead to overfitting the validation set. Dividing the training set ensures the model generalizes well.

5-Fold Cross-Validation Workflow

1. Split the resampled training set into 5 equal parts (folds).
2. For each parameter combination, train the model 5 times: train on 4 folds, and validate on the remaining 1 fold.
3. Average the Macro F1 validation score across all 5 folds.
4. Select the parameter combination that yields the highest average score.

Grid Search Parameters in Task 11

- **hidden_layer_sizes:** (32, 16) (2 layers) vs. (64, 32) (2 layers)
- **activation:** 'relu' (rectified linear) vs. 'tanh' (hyperbolic tangent)

Task 11: Pipeline Preprocessing & Scaling Isolation

Implementing a robust neural network workflow requires careful handling of data scaling statistics:

- **Feature Extraction:** Extract training features (X_{train}) and targets (y_{train}) from the oversampled dataset, and test features (X_{test}) and targets (y_{test}) from the split dataset.
- **Scaling Isolation:** Instantiates `StandardScaler` and calls `fit_transform(X_train)`, then calls `transform(X_test)`.
- **CRITICAL CONCEPT:** The test set X_{test} must **only** be transformed, not fit. Fitting on the test set leaks mean and standard deviation information from the evaluation partition, falsely inflating model diagnostics.
- **Model Grid Definition:** The search grid comprises 8 distinct neural network configurations ($2 \times 2 \times 2$ combinations of hidden sizes, activations, and weight decay bounds), optimized via Macro F1 metrics to weight classes equally.

Model Evaluation: Confusion Matrix & ROC Curves

We evaluate our multiclass model on the untouched test set using two diagnostic panels:

- Panel 1: Multiclass ROC Curves (One-vs-Rest):**
- Plots True Positive Rate (TPR) vs. False Positive Rate (FPR) for each class c treated as positive against all other classes.
 - Area Under the Curve (AUC) measures class separability.
 - An AUC close to 1.0 (e.g., > 0.90) confirms the network successfully distinguishes transient hosts from background galaxies.
- Panel 2: Confusion Matrix:**
- A 3×3 contingency table listing True Labels (rows) vs. Predicted Labels (columns).
 - Exposes where errors occur, e.g., if the model confuses CC-SNe hosts (Class 1) with Type Ia hosts (Class 2).

Section 4: Model Interpretability & OOP Assembly (Task 12)

The “Black Box” Problem in Neural Networks

- While neural networks achieve high classification accuracy, they consist of millions of nested matrix multiplications and non-linear boundaries.
- Unlike linear regression, you cannot simply look at a weight coefficient to see how important a feature is. The model acts as a **black box**.
- **Scientific Requirement:** In physics and astronomy, we do not just care about accurate predictions; we must understand **why** the model made a prediction (what physical features it prioritized).
- We resolve this using **Permutation Feature Importance**.

Permutation Feature Importance Theory

Let M be a trained classifier, $D = (\mathbf{X}, \mathbf{y})$ be the validation dataset, and s be the baseline score (Macro F1-score).

Breiman's Algorithm (2001)

For each physical feature j (e.g., Star Formation Rate):

1. **Shuffle Column:** Randomly shuffle the values of column j across the dataset. This breaks the physical correlation between feature j and the target label y while preserving the feature's distribution.
2. **Evaluate:** Calculate the model performance $s^{(j)}$ on this perturbed dataset.
3. **Compute Drop:** Define importance as the drop in performance:

$$I(j) = s - s^{(j)}$$

- If a feature is **critical**, shuffling it will degrade model performance, yielding a large $I(j) > 0$.
- If a feature is **irrelevant**, shuffling it has little effect, yielding $I(j) \approx 0$.

Task 12: Permutation Feature Importance in Action

In Task 12, we execute permutation calculations on the scaled test set:

- **Methodology:** We use `sklearn.inspection.permutation_importance` over 10 repeats (`n_repeats=10`) to compute stable mean and standard deviation importance scores.
- **Feature Rankings:** The calculations yield a ranked list of importances:
 1. $\log_{10} M_{\star}$ and $\log_{10} \text{SFR}$ show the highest mean drop in Macro F1-score.
 2. `ssfr` (Specific Star Formation Rate) follows as a secondary indicator.
 3. `redshift` has the least impact on F1 metrics.
- **Physical Inference:** Shuffling mass and SFR destroys the core physical signals of the host galaxy, which confirms that our MLP has successfully learned the underlying rate equations of stellar nucleosynthesis and supernova delay times.

Physical Interpretation of Model Importances

Why does the neural network prioritize certain features over others?

- **Mass ($\log_{10} M_{\star}$) and Star Formation Rate ($\log_{10} \text{SFR}$):** Shuffling these parameters results in the largest decrease in Macro F1-score.
 - **Mass dependence:** Type Ia supernovae are produced by white dwarfs in binary systems, which accumulate over time. Their rate scales with total stellar mass.
 - **SFR dependence:** Core-Collapse supernovae come from short-lived massive stars ($M \geq 8M_{\odot}$) that trace active star formation.
- **Specific Star Formation Rate ($\text{ssfr} = \log_{10} \text{SFR} - \log_{10} M_{\star}$):** This captures the relative activity of the galaxy, allowing the network to distinguish quenched ellipticals from active spirals.
- **Physical Validation:** The network has successfully discovered the fundamental physical rate correlations we programmed into the synthetic universe (SFMS and DTD power laws) without ever being shown the underlying equations!

Object-Oriented Software Design for Astronomy

In academic research, scientists often write long, disorganized scripts or notebooks. In production environments and sky-survey pipelines, individual models must be consolidated into a clean, modular structure.

- **Object-Oriented Programming (OOP):** Encapsulates variables and functions inside modular objects (classes).
- We wrap our entire Phase 1 and 2 models into a single, unified pipeline class:

StellarTraceEngine

- **Pipeline Components wrapped:**
 1. The Star Formation Main Sequence (SFMS) regression model (from Milestone 1) to predict baseline Star Formation Rates.
 2. The polynomial features transformer (PolynomialFeatures) to format input stellar mass.
 3. The fitted StandardScaler to normalize galaxy inputs.
 4. The optimized multiclass MLPClassifier (from Task 11).

Task 12: StellarTraceEngine Pipeline Assembly

The custom class `StellarTraceEngine` provides a modular and clean interface for host classification:

- **Constructor** (`__init__`): Accepts and registers four pre-fitted assets: `sfms_reg`, `sfms_poly`, `mlp_model`, and `scaler_model`.
- **Pipeline Prediction Method** (`predict_host_probabilities(log_mstar, redshift)`):
 1. Predicts baseline $\log_{10}$ SFR using the polynomial mass regression.
 2. Calculates specific star formation rate ($ssfr = \log_{10} \text{SFR} - \log_{10} M_{\star}$).
 3. Assembles the feature vector: $[\text{redshift}, \log_{10} M_{\star}, \log_{10} \text{SFR}_{\text{baseline}}, ssfr]$.
 4. Normalizes the vector using the pre-fitted `StandardScaler`.
 5. Executes `mlp.predict_proba` and maps outputs to a dictionary:
$$\{\text{'Background'} : P_0, \text{'CC-SN'} : P_1, \text{'Ia-SN'} : P_2\}$$
- **Generalization**: The user can pass any new galaxy physical parameters and obtain class assignments out-of-the-box.

Common Student Gotchas & Troubleshooting

Keep an eye out for these frequent student implementation bugs:

Data Leakage: Oversampling the entire dataset before train-test split. Warn them if they get a suspicious $F1 = 1.00$.

Data Scaling Leakage: Calling `scaler.fit_transform` on both the training and test sets. The scaler must only be fit on the training data:

`scaler.fit( $X_{\text{train}}$ )  $\rightarrow$  transform( $X_{\text{train}}$ ) and transform( $X_{\text{test}}$ )`

Otherwise, mean and standard deviation properties leak from the test set!

Oversampling Test set: Testing on oversampled points rather than the true, physical galaxy catalog.

MLP Convergence Warnings: If scikit-learn warns of non-convergence, increase `max_iter` to 1000 (as in Task 11) or decrease the learning rate.

What We Achieved Today

- Solved dataset class imbalances safely using stratified splitting and oversampling.
- Prevented data leakage to ensure models generalize to new telescope data.
- Programmed a feed-forward MLP neural network to predict supernova sites.
- Derived the Softmax logit gradient ($\frac{\partial \mathcal{L}}{\partial z_c} = P_c - y_c$) and regularized weight updates.
- Used Permutation Feature Importance to extract physical rules from our black-box model.
- Assembled our code into an OOP pipeline.

Homework: Complete the code sections in `stellar_trace_assignment2.ipynb` (Tasks 10–12) and ensure all evaluation plots render cleanly.

Session 3 Preview: Next session, we will model supernova rate equations